Директивы препроцессора

Препроцессор лучше всего рассматривать как отдельную программу, которая выполняется перед компиляцией. При запуске программы, препроцессор просматривает код сверху вниз, файл за файлом, в поиске директив. **Директивы** — это специальные команды, которые начинаются с символа `#` и НЕ заканчиваются точкой с запятой. Есть несколько типов директив, которые мы рассмотрим ниже.

Оглавление:

1. Директива #include
2. Директива #define
3. Макросы-объекты с текст_замена
4. Макросы-объекты без текст_замена
5. Условная компиляция
6. Область видимости директивы #define

Директива #include

Вы уже видели директиву #include в действии. Когда вы подключаете файл с помощью директивы #include, препроцессор копирует содержимое подключаемого файла в текущий файл сразу после строки с #include. Это очень полезно при использовании определенных данных (например, предварительных объявлений функций) сразу в нескольких местах.

Директива #include имеет две формы:

- `#include <filename>`, которая сообщает препроцессору искать файл в системных путях (в местах хранения системных библиотек языка C++). Чаще всего вы будете использовать эту форму при подключении заголовочных файлов из Стандартной библиотеки C++.
- `#include "filename"`, которая сообщает препроцессору искать файл в текущей директории проекта. Если его там не окажется, то препроцессор начнет проверять системные пути и любые другие, которые вы указали в настройках вашей IDE. Эта форма используется для подключения пользовательских заголовочных файлов.

Директива #define

Директиву #define можно использовать для создания макросов. **Макрос** — это правило, которое определяет конвертацию идентификатора в указанные данные.

Есть два основных типа макросов: макросы-функции и макросы-объекты.

Макросы-функции ведут себя как функции и используются в тех же целях. Мы не будем сейчас их обсуждать, так как их использование, как правило, считается опасным, и почти всё, что они могут сделать, можно осуществить с помощью простой (линейной) функции.

Макросы-объекты можно определить одним из следующих двух способов:

`#define идентификатор`

Или:

`#define идентификатор текст_замена`

Верхнее определение не имеет никакого `текст_замена`, в то время как нижнее — имеет. Поскольку это директивы препроцессора (а не простые стейтменты), то ни одна из форм не заканчивается точкой с запятой.

Макросы-объекты с текст_замена

Когда препроцессор встречает макросы-объекты с `текст_замена`, то любое дальнейшее появление `идентификатор` заменяется на `текст_замена`. `идентификатор` обычно пишется заглавными буквами с символами подчёркивания вместо пробелов.

Рассмотрим следующий фрагмент кода:

```
#define MY_FAVORITE_NUMBER 9

std::cout << "My favorite number is: " << MY_FAVORITE_NUMBER << std::endl;
```

Препроцессор преобразует вышеприведенный код в:

```
std::cout << "My favorite number is: " << 9 << std::endl;
```

Результат выполнения:

```
My favorite number is: 9
```

Мы обсудим это детально, и почему так не стоит делать, на следующих уроках.

Макросы-объекты без текст_замена

Макросы-объекты также могут быть определены без `текст_замена`, например:

```
#define USE_YEN
```

Любое дальнейшее появление идентификатора `USE_YEN` удаляется и заменяется «ничем» (пустым местом)!

Это может показаться довольно бесполезным, однако, это не основное предназначение подобных директив. В отличие от макросов-объектов с `текст_замена`, эта форма макросов считается приемлемой для использования.

Условная компиляция

Директивы препроцессора условной компиляции позволяют определить, при каких условиях код будет компилироваться, а при каких — нет. На этом уроке мы рассмотрим только **три директивы условной компиляции**:

- `#ifdef`
- `#ifndef`
- `#endif`

Директива #ifdef (сокр. от «***if def****ined*» = *«если определено»*) позволяет препроцессору проверить, было ли значение ранее определено с помощью директивы #define. Если да, то код между #ifdef и #endif скомпилируется. Если нет, то код будет проигнорирован. Например:

```
#define PRINT_JOE

#ifdef PRINT_JOE
std::cout << "Joe" << std::endl;
#endif

#ifdef PRINT_BOB
std::cout << "Bob" << std::endl;
#endif
```

Поскольку `PRINT_JOE` уже был определен, то строка `std::cout << "Joe" << std::endl;` скомпилируется и выполнится. А поскольку `PRINT_BOB` не был определен, то строка `std::cout << "Bob" << std::endl;` не скомпилируется и, следовательно, не выполнится.

Директива #ifndef (сокр. от «***if n****ot* ***def****ined*» = *«если не определено»*) — это полная противоположность к #ifdef, которая позволяет проверить, не было ли значение ранее определено. Например:

```
#ifndef PRINT_BOB
std::cout << "Bob" << std::endl;
#endif
```

Результатом выполнения этого фрагмента кода будет `Bob`, так как `PRINT_BOB` ранее никогда не был определен. Условная компиляция очень часто используется в качестве header guards (о них мы поговорим на следующем уроке).

Область видимости директивы #define

Директивы выполняются перед компиляцией программы: сверху вниз, файл за файлом. Рассмотрим следующую программу:

```
#include <iostream>

void boo()
{
#define MY_NAME "Alex"
}

int main()
{
    std::cout << "My name is: " << MY_NAME;

    return 0;
}
```

Несмотря на то, что директива `#define MY_NAME "Alex"` определена внутри функции boo(), препроцессор этого не заметит, так как он не понимает такие понятия языка C++, как функции. Следовательно, выполнение этой программы будет идентично той, в которой бы `#define MY_NAME "Alex"` было определено ДО, либо сразу ПОСЛЕ функции boo(). Для лучше читабельности кода определяйте идентификаторы (с помощью #define) вне функций.

После того, как препроцессор завершит свое выполнение, все идентификаторы (определенные с помощью #define) из этого файла — отбрасываются. Это означает, что директивы действительны только с точки определения и до конца файла, в котором они определены. Директивы, определенные в одном файле кода, не влияют на директивы, определенные внутри других файлов этого же проекта.

Рассмотрим следующий пример:

function.cpp:

```
#include <iostream>

void doSomething()
{
#ifdef PRINT
    std::cout << "Printing!";
#endif
#ifndef PRINT
    std::cout << "Not printing!";
#endif
}
```

main.cpp:

```
void doSomething(); // предварительное объявление функции doSomething()

int main()
{
#define PRINT

    doSomething();

    return 0;
}
```

Результат выполнения программы:

```
Not printing!
```

Несмотря на то, что мы объявили `PRINT` в main.cpp (`#define PRINT`), это все равно не имеет никакого влияния на что-либо в function.cpp. Поэтому, при выполнении функции doSomething(), у нас выводится `Not printing!`, так как в файле function.cpp мы не объявляли идентификатор `PRINT` (с помощью директивы #define). Это связано с header guards.